A correlated subquery is a subquery that depends on the outer query for its values.

Unlike a normal subquery, a correlated subquery references one or more columns from the outer query.

Think of it this way:

- Normal subquery: Inner query can execute independently.
- Correlated subquery: Inner query needs a value from the outer query.

Let's understand with examples.

1. Normal subquery vs correlated subquery

Consider an `employees` table:

Employee	Department	Salary
Amit	IT	50000
Rahul	IT	70000
Priya	IT	60000
Neha	HR	40000
Ravi	HR	50000

Normal subquery: Find employees earning more than the overall average salary.

SELECT *

FROM employees

```
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
);
```

The inner query:

```
SELECT AVG(salary)
FROM employees;
```

Can execute independently. It calculates one average salary, which the outer query uses.

Correlated subquery: Find employees earning more than their department's average salary.

```
SELECT e.employee,
       e.department,
       e.salary
FROM employees e
WHERE e.salary > (
    SELECT AVG(e2.salary)
    FROM employees e2
    WHERE e2.department = e.department
);
```

Look at this condition:

```
WHERE e2.department = e.department
```

Here:

- `e2.department` belongs to the inner query.
- `e.department` belongs to the outer query.

The inner query depends on the current employee's department.

Result:

Employee	Department	Salary
Rahul	IT	70000
Ravi	HR	50000

2. How does a correlated subquery execute?

Let's take Amit from IT.

The outer query considers Amit:

e.employee = 'Amit'

e.department = 'IT'

e.salary = 50000

The inner query effectively becomes:

SELECT AVG(salary)

FROM employees

WHERE department = 'IT';

IT average salary:

$$(50000 + 70000 + 60000)/3 = 60000$$

Comparison:

$$50000 > 60000$$

False. Amit is excluded.

Next, Rahul:

- Department = IT
- Inner query calculates IT average = 60000.
- 70000 > 60000 → True.

Rahul is included.

Execution concept

Outer query

Read an employee row

Inner correlated query

Calculate average for that employee's department

Comparison

Is employee salary greater than department average?

TRUE

Include employee

FALSE

Exclude employee

Conceptually, this is evaluated for each outer row. However, the database optimizer may transform the query, so it does not necessarily execute the inner query physically once per row.

3. Important correlated subquery examples

Example A: Find employees earning the highest salary in each department

```
SELECT e.employee,
    e.department,
    e.salary
FROM employees e
WHERE e.salary = (
```

```
    SELECT MAX(e2.salary)
    FROM employees e2
    WHERE e2.department = e.department
);
```

Result:

Employee	Department	Salary
Rahul	IT	70000
Ravi	HR	50000

The inner query calculates the maximum salary for the department of the current outer employee.

Example B: Find employees who have the same salary as another employee

```
SELECT e1.employee,
       e1.salary
FROM employees e1
WHERE EXISTS (
    SELECT 1
    FROM employees e2
    WHERE e2.salary = e1.salary
      AND e2.employee <> e1.employee
);
```

This uses `EXISTS`, which checks whether the inner query returns at least one row.

Example C: Find departments having at least two employees

```
SELECT d.department
FROM departments d
WHERE (
    SELECT COUNT(*)
    FROM employees e
    WHERE e.department = d.department
) >= 2;
```

Again, the inner query uses the outer query's department.

4. Correlated subquery with EXISTS and NOT EXISTS

These are especially important in SQL interviews.

Find customers who have placed at least one order:

```
SELECT c.customer_id,
       c.customer_name
FROM customers c
WHERE EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.customer_id = c.customer_id
);
```

Find customers who have never placed an order:

```
SELECT c.customer_id,
       c.customer_name
```

```
FROM customers c
WHERE NOT EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.customer_id = c.customer_id
);
```

`EXISTS` and `NOT EXISTS` are frequently used for checking relationships between tables.

5. Correlated subquery vs JOIN vs Window Function

The same requirement can often be solved in multiple ways.

For example, find employees earning more than their department average.

Approach	Technique
Correlated subquery	Inner query references outer department
JOIN	Join employees with department averages
Window function	`AVG(salary) OVER (PARTITION BY department)`

Window function solution:

```
SELECT employee, department, salary
FROM (
    SELECT e.*,
        AVG(salary) OVER (
```

```
            PARTITION BY department
        ) AS dept_avg
    FROM employees e
)
WHERE salary > dept_avg;
```

The window-function approach is often cleaner for this type of calculation.

6. Important interview points

1. A correlated subquery references columns from the outer query.
2. It can use `EXISTS`, `NOT EXISTS`, comparison operators, and aggregate functions.
3. It is useful for row-by-row logical comparisons and existence checks.
4. It is not necessarily slower than a JOIN; performance depends on the query and execution plan.
5. Indexes on correlated lookup columns can help performance.

7. Practice question

Consider these two tables:

Customers

customer_id	customer_name
1	Amit
2	Rahul
3	Priya

4	Neha

Orders

order_id	customer_id	amount
101	1	500
102	1	700
103	2	300
104	3	900

Question: Write a correlated subquery to find customers who have not placed any orders.

Which approach would you use?

NOT EXISTS

EXISTS

MAX()

Check answer

My suggestion for your SQL interview preparation: focus particularly on correlated subqueries with EXISTS / NOT EXISTS, finding department-wise maximums, and comparing correlated subqueries with window functions.